

HL1 通信协议

PC 上位机与 RX71M、RA8P 使用同一套私有帧。帧内载荷是 Protocol Buffers 36.2（2026-09-17）的 proto3 编码。

日文版见 [host_link_protocol_ja.md](#)。

模式来源：

- 帧：[protocol/host_link.h](#)、[protocol/host_link.c](#)
- 载荷：[protocol/host_link.proto](#)（`package renesas.hostlink`）
- PC 侧独立实现：[PC_Host/src/uHostLink.pas](#)，字节必须与上面的 C 实现以及 protoc 36.2 一致

1. 物理层

业务串口，115200 8N1，无校验，无流控。日志串口只输出文本，不走本协议。

板卡	业务串口	TX	RX
RX71M	SCI2	P50	P52
RA8P	SCI7	P809	P808

交叉连接：PC 的 TX 接板卡 RX，PC 的 RX 接板卡 TX，共地。上位机一次只打开一个 COM 口，接到其中一块板的业务串口。

2. 私有帧 HL1

多字节整数均为小端。载荷最长 192 字节，整帧最长 202 字节。

偏移	长度	字段	说明
0	1	魔数 0	固定 0xA5
1	1	魔数 1	固定 0x5A
2	1	版本	当前为 1
3	1	板卡号	见下表
4	1	标志	见下表
5	1	序号低 8 位	与 Envelope.sequence 的低 8 位相同
6	2	载荷长度	小端，不含本字段和 CRC
8	N	载荷	Envelope 的 protobuf 编码
8+N	2	CRC16	小端

板卡号：

值	含义
0	不限。两块板都接受
1	RX71M
2	RA8P

标志：

位	名称	含义
bit0	HL_FLAG_FROM_BOARD	1 表示板卡发出
bit1	HL_FLAG_CAN_FD	1 表示该板支持 CAN FD

PC 发出的帧：标志为 0。板卡号为 0、1 或 2。板卡只处理板卡号为 0 或等于自身编号的帧，其余丢弃且不回答。

板卡发出的帧：bit0 置 1。RA8P 同时把 bit1 置 1，RX71M 不置 bit1。

CRC

CRC-16/CCITT-FALSE。

- 多项式：0x1021
- 初值：0xFFFF
- 输入、输出均不反转
- 结果异或：0x0000
- 覆盖范围：版本字节到载荷末尾，不含魔数和 CRC 自身
- 线上顺序：低字节在前

CRC 错误的帧丢弃，不作为应答。

3. Protobuf 载荷

编译器：protoc 36.2。语法：syntax = "proto3"。包名：renesas.hostlink。消息名：Envelope。

两块 MCU 不链接 libprotobuf。C 与 Pascal 各自实现同一套 proto3 线格式。

值为 0 仍有意义的控制字段使用 optional，因此 0 也会出现在线上。状态量仍按 proto3 习惯省略 0。

未知字段必须跳过，不能因此判帧非法。CAN 数据超过 8 字节则整条 Envelope 视为非法。

3.1 Envelope

字段号	名称	类型	何时出现
1	sequence	uint32	非 0 时。PC 从 1 递增；板卡主动上报从 0x80000000 递增
2	command	Command	非 0 时
3	snapshot	Snapshot	状态应答或周期上报

字段号	名称	类型	何时出现
4	set_outputs	SetOutputs	设置 GPIO 输出
5	set_dac	SetDac	设置 DAC
6	set_pwm	SetPwm	设置 PWM
7	send_can	SendCan	PC 请求发 CAN
8	can_log	CanLog	板卡上报已发送的 CAN
9	ack	Ack	写操作的应答

一条载荷可以同时带 **ack** 与 **snapshot**，或只带其中之一。**send_can** 成功时，板卡先回 **ack**，再另发一帧 **can_log**。

3.2 Command

值	名称	方向	板卡行为
0	COMMAND_UNSPECIFIED	—	不作为请求
1	COMMAND_PING	PC → 板	回 Snapshot
2	COMMAND_SNAPSHOT	PC → 板	回 Snapshot
3	COMMAND_SET_OUTPUTS	PC → 板	写 GPIO，回 Ack
4	COMMAND_SET_DAC	PC → 板	写 DAC，回 Ack
5	COMMAND_SET_PWM	PC → 板	写 PWM，回 Ack
6	COMMAND_SEND_CAN	PC → 板	发 CAN，回 Ack；成功后再报 CanLog

3.3 Snapshot

字段号	名称	含义
1	tick	板卡周期计数，约每 500 ms 加 1
2	adc0	第一路 AD
3	adc1	第二路 AD
4	inputs	GPIO 输入，bit0–bit3
5	outputs	GPIO 输出，bit0–bit3
6	dac	当前 DAC 码，0–4095
7	board	BOARD_KIND_RX71M=1 ， BOARD_KIND_RA8P=2
8	can_fd	该板能否发 CAN FD。RX71M 省略此字段；RA8P 为 true
9	name	RX71M 或 RA8P1

3.4 控制消息

SetOutputs.mask: 0–15, bit0–bit3 对应四路输出。0 表示全关，仍必须编码。

SetDac.code: 0–4095, 12 位。0 仍必须编码。

SetPwm:

字段	范围	说明
channel	0–3	四路 PWM
duty_permille	0–1000	占空比，单位千分比。500 表示 50%

channel 与 duty 为 0 时仍必须编码。

SendCan / CanLog:

字段	说明
channel	0 或 1。0 也必须编码
id	11 位标准帧，0–0x7FF
data	1–8 字节。0 字节视为参数错误
fd	false 为经典 CAN，true 为 CAN FD。两种取值都编码
tx	仅 CanLog。当前上报的都是板卡发送

RX71M 收到 fd=true 时不发送，Ack 状态为不支持，detail 为 can fd unsupported。RA8P 按 CAN FD 帧发送。两块板的自发 CAN 仍是经典帧，标识符为 0x123 与 0x321，数据为 1 字节周期计数，500 kbit/s。

3.5 Ack

字段	说明
command	被应答的命令号
status	0 成功，1 不支持，2 参数错误
detail	ASCII。成功时省略
detail	含义
bad protobuf	载荷无法按 Envelope 解码
bad outputs	GPIO 掩码大于 15，或缺少字段
bad dac	DAC 大于 4095，或缺少字段
bad pwm	通道或占空比越界
bad can	通道、ID、长度不合法
can fd unsupported	RX71M 拒绝 CAN FD
can busy	发送邮箱忙
unknown command	命令号未定义

status 为 0 时，该字段按 proto3 省略。

4. 时序

建立连接前，板卡不在业务串口上发送二进制帧。PC 打开串口后发送 `COMMAND_PING`。板卡收到第一帧合法且板卡号匹配的请求后，置连接已建立，并开始周期上报。

动作	周期
板卡读串口	约 20 ms
板卡采样并上报 Snapshot、自发 CAN 的 CanLog	约 500 ms，且仅在已连接后
PC 读串口	50 ms
PC 在未收到应答时重发 PING	约 1 s
PC 在已连接后请求 SNAPSHOT	约 1 s

PC 主动请求使用的 sequence 从 1 递增，并写入帧序号的低 8 位。板卡应答沿用该 sequence。板卡主动上报使用独立序号，从 `0x80000000` 递增，绕回后仍从 `0x80000000` 起，避免与 PC 序号相混。

连接建立后，板卡不再用演示循环改写 GPIO、DAC、PWM。ADC、输入和自发 CAN 仍继续更新。

5. 两块板的差异

帧格式、命令号和字段号相同。差异只在能力。

项目	RX71M	RA8P
板卡号	1	2
名称	<code>RX71M</code>	<code>RA8P1</code>
CAN	经典 CAN，两路，500 kbit/s	CAN FD，两路，仲裁段 500 kbit/s
标志 bit1	0	1
Snapshot.can_fd	省略（即 false）	true
<code>fd=true</code>	Ack = 不支持	发送 CAN FD

6. PC 操作与报文

操作	发送
连接	<code>command=PING</code>
读取状态	<code>command=SNAPSHOT</code>
下发输出	连续三帧： <code>SET_OUTPUTS</code> 、 <code>SET_DAC</code> 、 <code>SET_PWM</code>
发送 CAN	<code>command=SEND_CAN</code> ，并填写 <code>send_can</code>

目标板选“自动”时，帧内板卡号为 0。选定某一块板时，只把帧发给该编号；另一块板会丢弃。